Buổi 8

API Testing và Quality Assurance

Viết test case kiểm chứng API, Tự động hoá test suite với Postman/Newman

Đo lường performance và quality metrics trên Flask

Mục tiêu

Viết test case & tự động hoá test suite

① Viết test cases cho endpoint

Có khả năng phân loại happy path / error path / edge case; đặt tên test theo hành vi quan sát được.

② Tạo automated suite với Postman/Newman

Collection có testscript, biến môi trường, chạy lại được từ CLI, sinh báo cáo.

Đánh giá performance & quality metrics

① Đo latency / error rate / RPS

Phân biệt được p50 / p95 / p99 và vì sao không dùng trung bình cộng.

② Đánh giá tính phù hợp của test plan

Biết khi nào dùng load test, stress test, soak test, spike test; đọc output Locust thành thạo.

Vì sao API test quan trọng?

Chuyển JSON số → chuỗi khi tách microservice

Giả sử một API đổi giá tiền từ số 3990 sang chuỗi "3990" khi tách service. Client mong đợi kiểu số có thể xử lý sai. Đây là ví dụ giả định về lỗi hợp đồng dữ liệu.

→ Schema validation có thể phát hiện lỗi kiểu dữ liệu.

Pagination khác nhau giữa version v1 và v2

Giả sử server đổi hasMore thành has_more, đồng thời cách đánh số trang giữa client và API khác nhau. Contract test có thể phát hiện thay đổi về cấu trúc đã công bố.

→ Contract test hỗ trợ phát hiện breaking change.

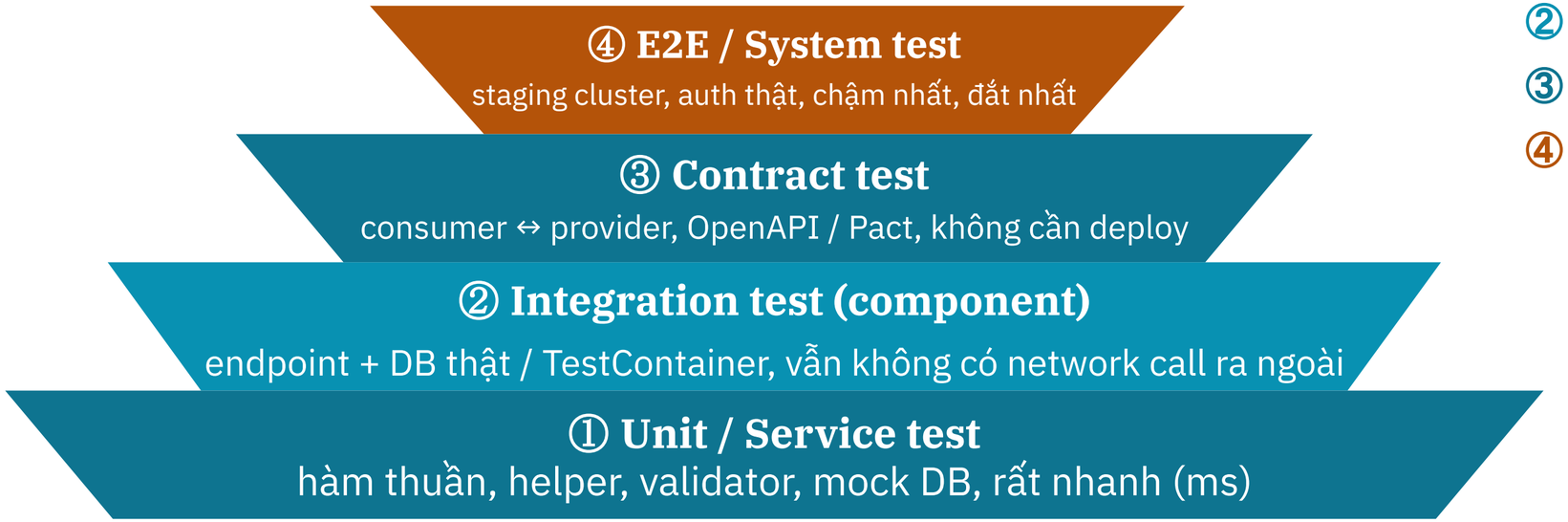
Endpoint "chạy được" nhưng chậm 6 giây

Giả sử GET /search trả đúng dữ liệu nhưng p95 đạt 6,2 giây dưới tải cao. Functional test không đánh giá được yêu cầu độ trễ; cần performance test.

→ Functional ≠ đủ; cần performance test.

Một API có thể vượt qua 100 functional test mà vẫn làm sập production — nếu thiếu schema validation, contract test, và performance test

4 mức độ kiểm thử API



Bắt được gì?

① Logic sai, branching, edge case

② Schema mismatch, query, transaction

③ Breaking change giữa team

④ Cấu hình infra, DNS, TLS

Công cụ phù hợp cho mỗi mức

Mỗi tầng test sẽ gắn với một (hoặc vài) công cụ cụ thể

TẦNG	CÂU HỎI TRẢ LỜI	CÔNG CỤ CHÍNH (PYTHON-FRIENDLY)	MẤT BAO LÂU	CHI PHÍ VẬN HÀNH
① Unit	Hàm `foo` cho kết quả đúng chưa?	`pytest` thuần, `unittest.mock`	< 1 s / test	**Rất thấp**
② Integration / Functional	Endpoint trả về status + body đúng theo OpenAPI?	`pytest` + `flask.test_client`, `jsonschema`	~100 ms / test	**Thấp**
③ Contract	Provider & consumer có đồng thuận về shape?	`pact-python`, OpenAPI diff tools, Spectral	~1–5 s / pact	**Trung bình**
④ E2E / Smoke	Một user journey chạy được trên staging?	Postman + Newman CLI, Playwright (Python)	~30 s / scenario	**Trung bình**
⑤ Performance	Hệ thống chịu được bao nhiêu RPS trước khi p95 vượt ngưỡng?	`locust`, k6 (JS), JMeter	~5–60 phút / plan	**Cao**

Mẹo thực tế: nếu một test fails ở tầng ②, fix ở tầng ② — không promote test đó lên ④. Test "đúng tầng" chạy nhanh, deterministic, ít flaky.

Books API

REST resource quản lý sách, viết bằng Flask 3.x + SQLAlchemy.

METHOD + PATH	MỤC ĐÍCH	STATUS CODE
`GET /books`	Liệt kê; hỗ trợ `?author=`, `?page=`	200
`GET /books/<id>`	Lấy chi tiết một sách	200 / 404
`POST /books`	Tạo sách — cần `title`, `author`	201 / 400 / 409
`PUT /books/<id>`	Cập nhật toàn bộ trường	200 / 404 / 400
`DELETE /books/<id>`	Xoá một sách	204 / 404

```
# list_books route
@app.get("/books")
def list_books(): q = Book.query
if a := request.args.get("author"): q = q.filter_by(author=a)
items = q.limit(20).all()
return jsonify([b.to_dict()
for b in items])
# factory
def create_app(cfg=None): app = Flask(__name__)
app.config.update( DB=cfg or"books.db")
db.init_app(app); db.create_all()
return app
```

Schema của Book

```
{ "id": 42, "title": "API Design Patterns", "author": "J.J. Geewax", "year": 2021, "isbn": "9781617295850" }
```

Marshmallow validate; unique `isbn`.

Thiết lập pytest cho Flask — conftest.py

tests/conftest.py định nghĩa ba fixture: app, client và sample_book. Mỗi test chỉ yêu cầu cái nó cần — pytest tự động inject.

```
# tests/conftest.pyimport pytest
from app
import create_app
from models
import db, Book
@pytest.fixture()
def app(): cfg = {"TESTING": True, "SQLALCHEMY_DATABASE_URI": "sqlite:///:memory:"}
app = create_app(config=cfg)
with app.app_context(): db.create_all()
yield app
with app.app_context(): db.session.remove(); db.drop_all()
@pytest.fixture()
def client(app):
return app.test_client()
@pytest.fixture()
def sample_book(app):
with app.app_context(): b = Book(title="API Design Patterns", author="Geewax", year=2021)
db.session.add(b); db.session.commit()
yield b.id
```

Vai trò của 3 fixture và 1 cấu hình

① `memory:` — CSDL RAM, biến mất khi test xong; không chạm CSDL phát triển.

② `test_client()` — request nội bộ WSGI, không mở socket.

③ `yield` — setup trước, teardown sau.

④ `sample_book` — mỗi test tạo data riêng, độc lập.

> **Quy ước đặt tên:** thư mục `tests/`, file `test_*.py`, hàm `test_*()`.

Test `GET /books`

Một "test tốt" phải trả lời được cả ba: status code (200), body shape (đủ trường, đúng kiểu), và business rule (lọc, phân trang).

Test chỉ kiểm tra status là test thiếu.

```
# tests/test_list_books.py
def test_list_books_empty(client): r = client.get("/books")
assert r.status_code == 200
assert r.content_type == "application/json"assert r.get_json() == []
# KHÔNG null, KHÔNG {}
def test_list_shape(client, sample_book): data = client.get("/books").get_json()
assert len(data) == 1 keys = {"id","title","author", "year","isbn"}
assert set(data[0].keys()) == keys
assert isinstance(data[0]["id"], int)
def test_filter_author(client, app):
from models
import db, Book
with app.app_context(): db.session.add_all([ Book(title="A", author="X"), Book(title="B",
author="Y"), Book(title="C", author="X")])
db.session.commit()
titles = [b["title"]
for b in client.get("/books?author=X").get_json()]
assert titles == ["A", "C"]
```

A. Protocol
status, Content-Type, cache header

B. Schema
đủ trường, đúng kiểu
Dùng `set ==` để so sánh đúng tập.

C. Business rule
lọc, sắp xếp, phân trang, rỗng

"Chỉ assert 200 không bắt được lỗi shape — nó bắt 'server sập', không bắt 'trả sai shape'."

Test 404, 400 và 401

Error path cần được kiểm thử riêng. Không phải mọi 5xx đều nên retry; chỉ retry khi phù hợp với ngữ nghĩa request và chính sách backoff.

```
# 404 — missing + 400 ID sai
def test_404(c): r = c.get("/books/9999")
assert r.status_code == 404
def test_400_letter(c): """abc → 400, không 500""" r = c.get("/books/abc")
assert r.status_code == 400# 400 — body validation
def test_400_no_title(c): r = c.post("/books", json={"author":"X"})
assert r.status_code == 400 body = r.get_json()
fields = {e["field"]
for e in body.get("errors", [])} assert"title"in fields
# 401 — chưa auth
def test_401_no_token(c):
assert c.delete( "/books/1").status_code == 401
```

Auth test — 401 vs 403

401 Unauthorized — chưa đăng nhập.

403 Forbidden — đã đăng nhập nhưng thiếu quyền.

Không trả cùng code — log audit mất phân loại.

```
def test_no_token_401(c):
assert c.delete("/books/1" ).status_code == 401
def test_other_user_403(c, auth): h = {"Authorization": f"Bearer {auth}"}
assert c.delete("/books/2", headers=h).status_code == 403
```

✕ Anti-pattern: trả 500 thay vì 4xx

```
@app.get("/books/<int:id>")
def get_book(id): b = Book.query.get(id)
return b.to_dict() # crash nếu None
```

```
if b isNone: return jsonify(code="NOT_FOUND"), 404
```

→ status 4xx có ngữ nghĩa rõ ràng, máy khách retry được kiểm soát.

Tip: Helper `assert_error()`

Tách riêng assertion vào helper function để nhiều test cùng dùng lại — ví dụ:

```
defassert_validation_error(r, field): assert r.status_code == 400
body = r.get_json() fields = {e["field"] for e in body["errors"]}
assert field in fields
```

POST / PUT / DELETE — test "vòng đời"

Endpoint thay đổi trạng thái cần xác nhận ba thứ: response (status + body), side-effect (CSDL thật thay đổi), và idempotency (lặp yêu cầu có cùng hiệu ứng dự kiến).

```
# ① POST /books — tạo + verify side-effect
def test_post_persists(client, app): payload = {"title":"DDIA",
"author":"Kleppmann", "year":2017}
r = client.post("/books", json=payload)
assert r.status_code == 201 bid = r.get_json()["id"]
# Side-effect — query CSDL thật from models
import Book
with app.app_context():
assert Book.query.get(bid).title == "DDIA"# POST thường không
idempotent; 409 chỉ khi có xung đột được API định nghĩa
assert client.post("/books", json=payload).status_code == 409# ② PUT
/books/<id>
— replace
def test_put_replaces(client, sample_book): r =
client.put(f"/books/{sample_book}", json={"title":"Renamed",
"author":"new",
"year":2025})
assert r.get_json()["title"] == "Renamed"
```

```
# ③ DELETE — idempotent enough
def test_delete_404_after(c, sb):
assert c.delete( f"/books/{sb}").status_code == 204
assert c.get( f"/books/{sb}").status_code == 404# Unknown ID → 404
(không 204)
def test_delete_unknown_404(c):
assert c.delete( "/books/9999").status_code == 404# Lặp lại: 204 rồi
404
def test_delete_safe_repeat(c): bid = c.post("/books",
json={"title":"T", "author":"A"} ).json()["id"]
assert c.delete(f"/b/{bid}") == 204
assert c.delete(f"/b/{bid}") == 404
```

Những chỗ "xanh giả"

- **Test quên assert side-effect.** Trả 201 nhưng insert CSDL fail; data biến mất sau 5 phút. Luôn verify bằng query lại.
- **Test phụ thuộc thứ tự.** Test 2 cần data của test 1. Mỗi test phải tạo data riêng.
- **Thiếu concurrent test.** Hai PUT đồng thời có thể overwrite nhau — dùng `threading` (Lab 4).

Validate response theo OpenAPI với jsonschema

Assert field-by-field thì "đúng hôm nay" — nhưng không phát hiện khi backend thêm field lạ hoặc đổi `int↔str`.

```
# openapi.yamlcomponents.schemas.Book:
type: object
required: [id, title, author] additionalProperties: false
properties: id:
type: integer
minimum: 1 title:
type: string minLength/maxLength: 1/200
author:type: string
year:
type: integer
minimum: 1450
maximum: 2100
isbn:
type: string pattern: "^\d{13}$"
```

```
# tests/test_schema.pyimport yaml, pytest
from jsonschema
import validate, ValidationError BOOK = yaml.safe_load( open("openapi.yaml")
).components.schemas.Book
def test_each_match(c, _):
for b in c.get("/books").json(): validate(b, BOOK)
def test_extra_blocked(c, _): body = c.get("/books").json()[0] body["secret"] = 1
with pytest.raise s(ValidationError): validate(body, BOOK)
def test_year_oob(c, app):
with app.app_context():
from models
import Book, db db.session.add(Book(year=3000))
db.session.commit()
with pytest.raise s(ValidationError): validate(c.get("/books" ).json()[0], BOOK)
```

Sự cố #1 — số → chuỗi

`year` là `"2021"`. Schema `type:integer` bắt.

Sự cố #2 — field bí mật

Backend thêm `internal_score`. `additionalProperties:false` chặn.

Sự cố #3 — đơn vị/kiểu

Schema kiểm tra kiểu/định dạng; kiểm tra đơn vị tiền cần ràng buộc nghiệp vụ riêng.

Lab 1 — Test Books API từ đầu

```
$ https://github.com/TrinhLK/lab08-starter.git
$ cd lab08-starter && python -m venv .venv
$ source .venv/bin/activate
$ pip install -r requirements.txt # flask, pytest, jsonschema
```

Bước 1, Viết 4 test

A **GET /books** khi CSDL rỗng trả về mảng rỗng (không phải null).

B **GET /books/<id>** với id không tồn tại trả về 404 + JSON chứa `"code":"NOT_FOUND"`.

C **POST /books** thiếu `title` trả về 400 + trong `errors` có field `"title"`.

D **Xoá rồi GET** cùng id — phải trả về 404.

Bước 2, Bonus

Thêm test `test_response_extra_field_blocked` dùng `jsonschema` — chứng minh backend không lộ `password_hash` (giả lập bằng cách add field lạ trong fixture).

```
$ pytest -v ======== 5 passed in 0.43s ========
```

Tiêu chí đạt: cả 4 test pass trong <2 giây, không in stack trace. Lab xong sớm được cộng 0.5đ vào bài tập về nhà.

Nộp bài: nén folder `tests/` + `pytest.ini` vào file `lab01_<mssv>.zip` qua LMS trước 23:59 Chủ nhật. Commit cũng được nhưng LMS là kênh chính thức.

Contract testing — khi hai team cần đồng thuận

Trong microservice, *integration test end-to-end* rất đắt: cần spin cả cluster, fix lỗi mất hàng giờ, và mỗi lần đổi một field là phải đẩy ngược chuỗi. Contract test cắt ngắn: kiểm tra hai bên *đồng ý về shape* mà không cần chạy cả hệ thống.

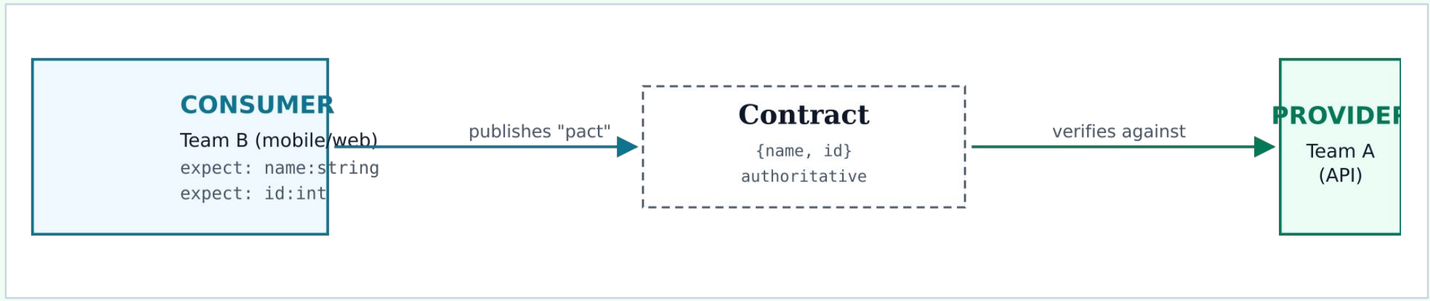
✕ Vấn đề nếu chỉ dùng E2E

Team A đổi `user.name` thành `user.fullName`. Team B không biết.

Nếu không có kiểm tra hợp đồng sớm, CI hoặc staging của team B có thể phát hiện lỗi muộn. VD: chỉ 1 test có cover case này — và nó chạy ở staging sau microservice khác.

Chi phí khắc phục lỗi thường tăng khi phát hiện muộn; không áp dụng một hệ số cố định cho mọi dự án.

√ Cách contract test hoạt động



Khi Team B đẩy test, nó *xuất bản* một bản ghi kỳ vọng. Provider Team A chạy lại bản ghi đó — nếu API sai, fail ngay. Cả hai team không cần share repo hay deploy cùng thời điểm.

"Contract test không thay thế functional test — nó thay thế sự cần thiết phải chạy production cluster để biết API còn đúng."

Nguồn: Marcin Grzejszczak, Mastering Spring Cloud Contract (Packt, 2020) §3; Pact docs (pact.io).

Contract testing, cách tiếp cận A

OpenAPI là "single source of truth"

Cách đơn giản nhất để "có contract" với REST API là viết OpenAPI spec *trước khi* viết code. Spec vừa là tài liệu, vừa sinh test + code + doc cùng lúc.

openapi.yaml — nguồn duy nhất

"Một file, bốn sản phẩm." Đổi spec → cập nhật cả test, doc, server stub và client generator.

① Test client (pytest)

`generate từ spec`
`openapi-python-client`

② Docs (Redoc)

`spec → HTML page`
`auto-update`

③ Schema validation

`jsonschema loads từ`
`components.schemas`

④ Postman / stub

`spec → import vào`
`Postman collection`

Spectral — lint OpenAPI

Stoplight Spectral đọc `.spectral.yml` rule-set; fail build nếu spec thiếu `operationId`, status code thiếu schema, hay `description` quá ngắn.

> **Nên tích hợp vào:** pre-commit hook (chặn local), CI check (chặn merge), và CD dry-run (chặn release).

Contract testing, cách tiếp cận B

Schema-based vs Consumer-driven (Pact)

Schema-based tin vào một OpenAPI "chuẩn"; **Consumer-driven** cho rằng nhà tiêu dùng quyết định field nào thực sự được dùng. Lựa chọn sai → test quá rộng (catches nothing) hoặc quá hẹp (catches everything).

Cách A

Schema-based (OpenAPI/Spectral)

+ OpenAPI đã có sẵn, không cần viết thêm.
+ Có thể enforce nhiều rule (lint) mà không cần test.
+ Phù hợp public API — provider chủ động công bố.

– Không phân biệt field "có dùng" vs "có tồn tại". Nếu provider thay đổi kiểu của một trường không được schema kiểm tra, test có thể vẫn pass; schema đầy đủ có thể phát hiện thay đổi kiểu.

– Thường fail ở giai đoạn "đổi mới" — provider phải đợi consumer "theo kịp".

Dùng khi: một team làm API, một team consume; spec công bố trước, không ai quyết shape một mình.

Cách B

Consumer-driven (Pact)

+ Mỗi consumer tự publish một "pact" — chỉ chứa field nó thực sự gọi.
+ Đổi field không ai dùng → không break.
+ Cho (micro)service có nhiều consumer với nhu cầu khác nhau.

– Cần Pact Broker để chia sẻ file pact giữa team — chi phí vận hành.
– Mỗi consumer viết 1 pact — tổng số file pact có thể bùng nổ.

Dùng khi: nhiều team consume cùng một API; mỗi team có use-case riêng; không có OpenAPI đáng tin cậy.

Lời khuyên: dùng Spectral nếu đã có OpenAPI; cân nhắc Pact khi cần kiểm tra kỳ vọng riêng của nhiều consumer.

Lab 2 — Lint OpenAPI bằng Spectral

File `openapi.yaml` trong starter có 3 lỗi cố ý. Hãy dùng Spectral để phát hiện, sửa, rồi commit.

Fix 3 "lỗi" trong code

1. **`GET /books`** thiếu `operationId`.
2. **DELETE /books/{id} khai báo body cho response 204 (không phù hợp).**
3. **Book.year có type: string (khác mô hình Book dự kiến: integer; cần kiểm tra thêm bằng rule/test).**

Sau khi sửa 3 lỗi trên, thêm một **custom rule**: response 200 phải có nhiều hơn 0 trường trong schema.

> Nếu spec hợp lệ Spectral-pass nhưng code Flask trả sai field — test nào bắt được? Bạn sẽ viết gì?

```
# Cài đặt & chạy
$ npm install -g @stoplight/spectral-cli # Lint mặc định (OAS rule-set)
$ spectral lint openapi.yaml Kết quả lỗi phụ thuộc ruleset; không mặc định bắt đủ 3 lỗi
# Sau khi sửa cả 3, chạy lại
$ spectral lint openapi.yaml --ruleset .spectral.yml ✓ No problems
# Tích hợp vào pre-commit hook# .pre-commit-config.yamlrepos: -
repo: https://github.com/stoplightio/spectral hooks: - id: spectral args: [--ruleset, .spectral.yml]
# Hoặc chạy trong GitHub Actions - run: npx @stoplight/spectral-cli \ lint openapi.yaml
```

Yêu cầu

- File `openapi.yaml` được sửa đủ 3 lỗi — Spectral lint trả về 0 problem.
- File `.spectral.yml` có ít nhất 1 custom rule — giải thích nó catch case gì.
- Chạy bằng `spectral lint <file> --ruleset .spectral.yml`; nộp output log kèm.

Cấu trúc một Postman Collection

Postman organize API call theo **collection** → **folder** → **request**. Mỗi request có URL, method, headers, body, pre-request script và test script. Tổ chức tốt giúp 200 test chạy thành 1 click.

📁 **books-api/ (collection)**

- 📁 auth/
  - POST /auth/login— lấy token
  - POST /auth/refresh
- 📁 books/
  - GET /books
    - `test: status 200, body là array`
  - GET /books/{id}
    - `test: có đủ 5 trường`
  - POST /books
    - `test: id là số, >0`
  - PUT /books/{id}
  - DELETE /books/{id}
- 📁 error-cases/
  - POST /books thiếu title
    - `test: 400 + "VALIDATION_ERROR"`
  - GET /books/9999
    - `test: 404 + "NOT_FOUND"`

Cấu trúc 3 cấp (collection → folder → request) giúp chạy theo *folder* hoặc *request* riêng biệt.

Một request gồm 6 phần

1. **URL + Method** — `{{base_url}}/books/{{book_id}}`
2. **Params** — query string qua tab "Params", không ghép vào URL.
3. **Headers** — `Authorization: Bearer {{token}}`
4. **Body** — JSON, form-data, raw,...
5. **Pre-request script** — JS chạy *trước* khi gửi (tạo token, set timestamp...)
6. **Tests** — JS assertions sau khi nhận response.

> **Mẹo:** đặt tên request kiểu "GET /books returns 200 với array rỗng" sẽ giúp test output dễ đọc — đừng gọi là "test 1".

Mỗi folder có thể có **folder-level auth**, **folder-level pre-request** — sinh ra một API demo trong <30 phút.

Ví dụ, Test scripts

Viết assertion với pm.test API

Mỗi `pm.test(name, fn)` là một assertion block; `pm.response` và `pm.expect` là helper. Test fail sớm dừng assert trong cùng block (trừ khi bọc `try`).

```
// Tests tab của GET /books/<id>// ① Status helper pm.test("status 200", () => {
pm.response.to.have.status(200); });
// ② Schema pm.test("có 5 trường required", () => { const b = pm.response.json();
pm.expect(b).to.have.all.keys( "id","title","author", "year","isbn"); }); pm.test("id là số
dương", () => { const id = pm.response.json().id; pm.expect(id).to.be.a("number");
pm.expect(id).to.be.above(0); });
// ③ JSON Schemaconst schema = { type: "object", required: ["id","title"],
additionalProperties: false }; pm.test("schema match", () => {
pm.response.to.have.jsonSchema(schema); });
// ④ Lưu state pm.collectionVariables.set( "last_book_id", pm.response.json().id);
```

6 best practice

1. Một `pm.test` = một khẳng định fail riêng.

2. Tên test mô tả hành vi — đừng "Test 1".

3. `collectionVariables.set` để truyền data.

4. Đừng viết logic lớn trong script — tách helper.

5. `responseTime` cho smoke; Service Level Objective (SLO) gate ở CI.

6. `environment.has` chỉ dành guard.

> **Mẹo:** với tập test lớn, dùng `pm.sendRequest` cho sub-request trong cùng script.

Cấu hình hoá test

Environment, variable & data file

Một collection chạy trên nhiều environment (local, staging, prod) và *nhiều lần* với data khác. Chia layer: **variable** cho config, **data file** cho input, **collection Variable** cho state.

① Environment

Config cho một môi trường — URL gốc, secret key, feature flag.

```
{ "id": "staging", "values": [ { "key": "base_url", "value": "https://api-staging.books.io" }, { "key": "token", "value": "eyJhbGciOiJI..." } ] }
```

② Data file (CSV/JSON)

Cùng collection chạy nhiều lần với input khác — data-driven.

```
# data.csv (UTF-8, no BOM) title,author,year DDIA,Kleppmann,2017 SICP,Abelson,1996
```

Newman `-d data.csv` chạy 3 dòng, mỗi dòng một run.

③ Collection variables (state)

State sinh ra trong run (id sách vừa tạo) — dùng `pm.collectionVariables.set(...)`.

Workflow mẫu: tạo rồi xoá

```
// Request A — POST /booksconst body = pm.response.json();
pm.collectionVariables.set( "book_id", body.id);
// Request B — DELETE /books/{{book_id}}// Test Request B
pm.test("xoá thành công", () => {
pm.expect(pm.response.code).to.equa
l(204); });
```

Hai request chạy tuần tự; B "nhớ" id từ A. Nếu A fail → B vẫn chạy để debug dễ.

Sai lầm hay gặp

- **Hard-code URL.** Mỗi request gốc localhost — sang staging phải sửa cả file.
- **Global thay vì variable.** Global rò rỉ giữa collection, khó debug hơn.
- **CSV có BOM.** Newman đọc sai cột — dùng UTF-8 *không BOM*.

Tích hợp CI/CD

Newman — chạy collection từ terminal

Newman là CLI companion của Postman — chạy 1 collection như một file. Cài qua npm; dùng cho CI hoặc pre-push hook.

```
# Cài đặt (Node ≥ 18)
$ npm install -g newman
# Chạy collection + environment + data
$ newman run books-api.postman_collection.json \ -e staging.postman_environment.json \ -d data.csv
--folder books
```

```
# .github/workflows/api-tests.ymlon: [push, pull_request] jobs: newman: runs-on: ubuntu-latest steps:
- uses: actions/checkout@v4 - uses: actions/setup-node@v4 with: { node-version: "20" } - run: docker
compose up -d api - name: Run Newman run: newman run collection.json -e env.json -r cli,junit
--reporter-junit-export junit.xml - if: always() uses: actions/upload-artifact@v4
```

Flag thường dùng

```
-e <file> environment
-d <file> data CSV/JSON
-n <num> iterations
--folder 1 folder
--bail fail đầu tiên
-r cli/json/junit/html
```

Vì sao Newman hợp CI

- Exit code 0 = pass, ≠0 = fail; CI log & stage.
- Xuất JUnit XML — mọi CI hiển thị.
- Không GUI; chạy container <200 MB RAM.
- Cùng collection → cùng behavior local + CI.

Lưu collection vào git — version control quan trọng hơn "Postman cloud".

Newman 6.x cần Node ≥ 18; docs: learning.postman.com/docs/postman-cli.

Output & CI gate

Reporter và exit code

Mỗi reporter trả về một output *khác nhau*: **CLI** cho người đọc, **JUnit** cho CI, **HTML** cho stakeholder, **JSON** cho tool khác đọc. Exit code mặc định của Newman là 1 nếu bất kỳ test fail.

4 reporter tích hợp sẵn

CLI
Màu sắc, có progress bar, mặc định cho developer.

JUnit
XML cho GitHub Actions / GitLab / Jenkins.

HTML
Một file HTML self-contained; report đầy đủ.

JSON
Structured; tool bên ngoài đọc để dashboard.

Cú pháp:

```
newman run collection.json -r cli,junit,html \
--reporter-junit-export reports/junit.xml \
--reporter-html-export reports/report.html
```

Exit code & status

exit 0
Mọi test pass.

exit 1
Một hoặc nhiều test fail (cả assertion hoặc network error).

Flag `--successExitCode` cho phép đổi exit code (mặc định 0); `--bail` dừng ngay khi có 1 fail.

Bẫy: nếu một assertion bị *error* (không phải fail), test runner vẫn coi là fail. Cần test "pass-fast" để phân biệt lỗi do test vs lỗi do API.

```
# ví dụ junit.xml (một phần) <testsuite name="books-api" tests="5"
failures="1" errors="0"> <testcase name="GET /books
return s 200"
class name="books.list"/> <testcase name="POST /books validators
return 400"
class name="books.create"> <failure type="AssertionError"> expected
status
400, got 200 </failure> </testcase> </testsuite> # CI rule: tests > 0 mới
hợp lệ;# failures hoặc errors ≥1 → pipeline đỏ
```

Lab 3 — Build Newman-runnable collection

Ban đầu có `books-api.postman_collection.json` (chỉ happy paths). Hãy thêm 2 request error + 2 test script bắt buộc cho mỗi request, rồi chạy bằng Newman.

A Thêm folder `error-cases/` với 2 request: **`POST /books`** thiếu `title` và **`GET /books/9999`**.

B Mỗi request có ≥2 `pm.test`: 1 cho status code, 1 cho body (code/message).

C Dùng **`collectionVariable`** lưu `book_id` từ POST, dùng cho DELETE request.

D Chạy bằng Newman với `-r cli,junit`; nộp file `junit.xml`.

```
# Cài Newman (cần Node)
$ npm install -g newman
# Chạy (đảm bảo Books API đang chạy)
$ newman run books-api.postman_collection.json \ -e
staging.postman_environment.json \ -r cli,junit \
--reporter-junit-export junit.xml
# Output kỳ vọng (rút gọn)→ books → GET /books
..... 200 OK → GET /books/{id}
..... 200 OK → POST /books
..... 201 Created → PUT /books/{id}
..... 200 OK → DELETE /books/{id}
..... 204 No Content → error-cases → POST
/books (missing title) ..... 400 ✗ FAIL → GET
/books/9999 ..... 404 ✗ FAIL # Nếu fail,
output× AssertionError: expected "VALIDATION" but got
"BAD_REQUEST" at eval (eval at ...)
```

Yêu cầu

- Collection chạy độc lập với code Flask (tức là API có thể chạy ngay trong máy, *không* cần dùng Postman UI mở).
- 5 happy-path test + 2 error-path test qua Newman: tổng cộng <10 giây, JUnit file có >= 7 testcase.
- `book_id` lưu & dùng đúng qua collectionVariable.
- Một shell script hoặc README ghi lại command chạy — người khác đọc là chạy được ngay.

Quality metrics — đo cái gì?

Google SRE Book định nghĩa bốn chỉ số "Golden Signals" cho mọi service. Đặt mục tiêu *hợp lý* theo từng endpoint — không phải endpoint nào cũng có cùng SLO.

① Latency
Thời gian đáp

Phân biệt **success latency** (response 200/201) với **failure latency** (response 4xx/5xx) — backend chậm khi lỗi thường tệ hơn happy path.

`p50 / p95 / p99 — *không* dùng mean`

② Traffic
Lưu lượng

RPS (request per second) cho HTTP API; **QPS** cho query; **concurrent users** cho góc nhìn người dùng.

`cao điểm / trung bình / tối thiểu 1 ngày`

③ Errors
Tỉ lệ lỗi

Tổng `5xx / total` chia làm 3 loại: do dữ liệu, do timeout, do exception chưa lường.

`Ngưỡng lỗi cần đặt theo SLO và ngữ cảnh dịch vụ`

④ Saturation
Mức chiếm dụng

Bao xa service đến giới hạn: **CPU**, **RAM**, **conn pool**, **queue depth**.

`Không có ngưỡng saturation chung cho mọi tài nguyên`

Vì sao dùng p95thay vì `mean`?

Mean bị "kéo méo"

100 request, 99 cái = 100ms, 1 cái = 30s (timeout nặng): mean = 399ms — trông vẫn ổn.

`mean = 399 ms`

Percentile nhìn thấy outlier

Cùng scenario: p50 = 100ms (gần như mọi người), p95 = 100ms, max = 30s; p99 phụ thuộc cách tính percentile trên mẫu nhỏ.

`p95 = 100 ms, max = 30 000 ms`

Chọn percentile cho SLO theo tác động người dùng và lượng truy cập. p50 mô tả trung vị độ trễ, không phải throughput.

Phân loại performance test

5 loại performance test — chọn đúng loại

Mỗi loại trả lời một câu hỏi khác nhau. "Performance test" có thể hiểu theo nhiều cách — xác định rõ *mục tiêu* trước khi chạy

Load test

Mục tiêu: kiểm tra hệ thống chạy ổn ở tải dự kiến.

Pattern: 100 user, mô phỏng 30 phút.

```
Q: khả năng đáp ứng
"bình thường"?
```

Stress test

Mục tiêu: đẩy qua ngưỡng bình thường, quan sát "graceful degradation".

Pattern: 100→500 user, giữ 10 phút.

```
Q: khi nào hệ thống vỡ?
```

Spike test

Mục tiêu: đột biến tải trong thời gian rất ngắn.

Pattern: 0→1000 user trong 10 giây.

```
Q: auto-scale có kịp?
```

Soak test

Mục tiêu: tải trung bình kéo dài (memory leak, file handle).

Pattern: 50 user, 6–24 giờ.

```
Q: có leak sau đêm?
```

Breakpoint

Mục tiêu: tăng dần đến khi vỡ để tìm *capacity ceiling*.

Pattern: ramp 10→2000 user.

```
Q: max throughput?
```

Kịch bản mẫu, sản phẩm thương mại điện tử

Black Friday

Spike test khi mở sale — auto-scale có kịp 0→5000 RPS trong 30 giây?

Migrate DB

Stress test trên cụm mới — buffer còn 30% khi gấp đôi tải hay không?

Trước release

Soak test đêm — connection pool, RAM, logrotate có hoạt động qua 8 giờ?

Lưu ý: không để stress test làm sập hệ thống — cần có *circuit breaker* hoặc cho phép IP test riêng để rollback. Trong staging cluster, "phá" OK; trên production thì *không*.

Metric tổng hợp

Apdex — gộp latency thành 1 số 0–1

Đặt ngưỡng **T** = thời gian user "hài lòng" (ví dụ 200ms). Response < T → Satisfied, T–4T → Tolerating, > 4T → Frustrated.

Satisfied	Tolerating	Frustrated
`0 ≤ t ≤ T` (ví dụ: 0–200ms) weight: **1.0**	`T < t ≤ 4T` (ví dụ: 200–800ms) weight: **0.5**	`t > 4T` (ví dụ: > 800ms) weight: **0**

Công thức

```
Apdex = (Satisfied + Tolerating/2) / Total
```

Một service trả 60 request < 200ms, 30 request 200–800ms, 10 request > 800ms (tổng 100): Apdex = (60 + 15) / 100 = **0.75**.

Ngưỡng phổ biến

0.94 – 1.00	**Excellent**
0.85 – 0.93	**Good**
0.70 – 0.84	**Fair**
0.50 – 0.69	**Poor**
< 0.50	**Unacceptable**

Hạn chế của Apdex

① Chọn T sai → Apdex "ảo". T phải phản ánh user expectation, không phải capability hiện tại.

② Với endpoint phụ (admin, batch) T có thể = 5 giây — gộp chung sẽ "kéo" Apdex xuống.

Chuẩn Apdex có ý nghĩa khi theo dõi *qua thời gian*: 0.85 → 0.82 là tín hiệu thoái hoá dù cả hai "Good".

Lựa chọn công cụ

Locust vs k6 vs JMeter

Locust

Ngôn ngữ: Python (chỉ Python)
Phong cách: code-first, không cần DSL

+ Python thuần → debug & viết test scenario quen thuộc
+ Web UI real-time (đẹp hơn JMeter)
+ Distributed mode chạy trên nhiều máybằng `--master/--worker`

– Chỉ Python — team JS/Java khó sửa
– Learning curve cho người chưa code Python

Dùng khi: team Python, cần viết test scenario phức tạp; *không* cần test 100k+ RPS trên 1 máy.

Grafana k6

k6

Ngôn ngữ: JavaScript (ES6)
Phong cách: Go runtime + script JS

+ Single binary, hiệu năng cao (Go runtime)
+ Threshold-as-code tích hợp
+ Grafana Cloud ecosystem tốt

– Phải học JS + module system riêng
– Reuse code Python khó

Dùng khi: team quen JS, cần throughput lớn, muốn CI gate trực tiếp trong test script.

Apache

JMeter

Ngôn ngữ: GUI + BeanShell/Groovy
Phong cách: drag-and-drop, XML config

+ Hỗ trợ nhiều protocol (HTTP, JDBC, LDAP, MQTT...)
+ Plugin ecosystem rộng
+ Được dùng phổ biến trong bank/enterprise (legacy)

– GUI nặng, kém "developer-friendly"
– JMX XML khó review trong git
– Khởi động chậm (~2 GB RAM)

Dùng khi: test nhiều protocol khác HTTP; team đã quen JMeter nhiều năm.

Locust "Hello World" cho Flask API

Một locustfile Python định nghĩa (1) `HttpUser` class và (2) `@task` methods. Locust sinh user ảo, mỗi user chạy random task theo `weight`.

```
# locustfile.pyfrom locust
import HttpUser, task, between
import random
class BooksUser(HttpUser): wait_time = between(1, 3)
host = "http://localhost:5000"@task(3)
def list_books(self): self.client.get("/books", name="GET /books")
@task(2)
def read_one(self): bid = random.randint(1, 50)
with self.client.get( f"/books/{bid}", catch_response=True) as r:
if r.status_code == 404: r.success()
@task(1)
def create(self): payload = {"title":"load-test", "author":"locust"} self.client.post("/books",
json=payload, name="POST /books") # Chạy từ terminal $ locust -f locustfile.py -u 50 -r 5 -t 60s $
locust -f locustfile.py # mở UI
```

5 khái niệm cần nắm

① **HttpUser** — 1 user ảo; loop các task.

② **@task(weight)** — weight cao → hay gặp; 3:2:1 = 50%/33%/17%.

③ **wait_time** — giả lập nghỉ; thiếu → DOS.

④ **catch_response** — tự quyết success/fail.

⑤ **name="..."** — gộp URL pattern.

Mô phỏng hành vi thật

Locust — user flow có state & phụ thuộc

Một user journey: đăng nhập → tạo → cập nhật → xoá. Nếu đăng nhập fail, các bước sau phải skip đúng.

```
# locustfile_realistic.pyfrom locust
import HttpUser, task, between
import random
class User(HttpUser): host = "http://localhost:5000" wait_time = between(2, 5)
def on_start(self): r = self.client.post("/auth/login", json={"user":"l", "pwd":"s"})
self.token = r.json().get( "token")
if r.ok elseNone@task(3)
def browse(self): self.client.get("/books")
@task(1)
def read(self): if not self.token:
return bid = random.randint(1, 100)
h = {"Authorization": f"Bearer {self.token}"} self.client.get(f"/books/{bid}", headers=h)
@task(1)
def admin_crud(self): ifnot self.token:
return h = {"Authorization": f"Bearer {self.token}"}
r = self.client.post("/books", json={"title":"x"}, headers=h)
if r.status_code == 201: self.client.delete( f"/books/{r.json()['id']}", headers=h)
```

① **Task weight** — ưu tiên tần suất: 3:1:1.

② **on_start/on_stop** chạy 1 lần / user — login + cleanup.

③ **State per-user**: không share giữa user, dễ race condition.

> **Quan trọng:** weight là xác suất; admin thật nên dùng class riêng với `fixed_count`.

Output & phân tích

Đọc output Locust — 5 chỉ số quan trọng nhất

Một run Locust out ra 4 loại thông tin: **request count**, **latency distribution**, **throughput**, **failures**. Đọc tuần tự để biết "có vấn đề ở đâu".

```
# Output mẫu sau 60 giây, 50 userTypeName# reqs# failsAvgp95p99RPS
-------- ----------------------- ------ ------- ------- -------- -------
-------
GET /books 12840(0.0%) 42ms 98ms 145ms 21.4GET /books/{id} 84523(2.7%)
68ms 180ms 412ms 14.1POST /books 4323(0.7%) 125ms 290ms 680ms 7.2PUT
/books/{id} 2181(0.5%) 110ms 280ms 510ms 3.6DELETE /books/{id} 2160(0.0%)
38ms 85ms 120ms 3.6 -------- ----------------------- ------ -------
------- -------- ------- ------- Aggregated299527(0.9%) 76ms 198ms 512ms
49.9Response time percentiles (truncated)[update][copy][expand] Percentage
50%66%75%80%90%95%98%99%99.9%99.99% Value(ms)
3862881181651983405128801100Error report # occurrences Error
----------------- ---------------------------------------- 23
HTTPError("404 Client Error") for url: /books/{id} 3 HTTPError("409
Conflict") for url: /books 1 ReadTimeout
```

> **Cảnh báo:** trong ví dụ trên, 23 lần 404 trên `GET /books/{id}` có thể là *hành vi mong đợi* (id ngẫu nhiên, một số không tồn tại) — cần phân biệt với lỗi 500.

Đọc báo cáo như thế nào

① **Cột # fails** tỉ lệ cao (>1%) cho endpoint quan trọng → fail rate threshold failed.

② **p99 >> p95** (ví dụ p99=512, p95=198) — có một tail dài 1% user rất chậm. Check GC, lock, slow query.

③ **RPS không tăng tuyến tính** khi tăng user → saturations (lab 4 sẽ quan sát).

④ **Error report cuối** — phân loại: lỗi 4xx (client bug), 5xx (server bug), Timeout (network/load).

⑤ **Percentile chart** ở cuối — đường tăng nhanh quá ở 99% → bimodal distribution (2 loại request).

Lab 4 — Load test Books API với Locust

```
$ python -m app --port 5000 # Một terminal khác $ pip install locust
```

Bước 1, Tạo locustfile

thêm 1 task `search_by_year(year)` có 1% fail rate (giả lập 500).

Bước 2, Ba kịch bản chạy

```
# Kịch bản A — baseline (nhẹ)
$ locust -f locustfile.py --headless \
-u 10 -r 2 -t 30s --html reports/A.html

# Kịch bản B — load test thông thường
$ locust -f locustfile.py --headless \
-u 50 -r 5 -t 60s --html reports/B.html

# Kịch bản C — spike test
$ locust -f locustfile.py
--headless \ -u 200 -r 50 -t 30s --html reports/C.html
```

Nộp: nén `locustfile.py` + 3 file HTML trong `reports/` + `analysis.md`.

Bước 3, Phân tích

Trong nộp bài, viết file `analysis.md` trả lời:

1. Trong 3 kịch bản, kịch bản nào có p95 cao nhất? Vì sao?

2. Có endpoint nào fail rate tăng theo số user? Đoán nguyên nhân dựa trên code.

3. Nếu prod cần SLO p95 < 300ms, kịch bản nào đạt? Đề xuất baseline cho CI gate.

Mẹo: vẽ *p95 theo số user* từ 3 file HTML: đường p95 thường *tăng phi tuyến* sau một ngưỡng.

Câu hỏi bonus: nếu connection pool giới hạn 5, điều gì xảy ra khi 200 user truy cập đồng thời? Làm thế nào để xác nhận bằng Locust?

5 bài học từ production

Năm tình huống minh họa các rủi ro thường gặp. Các giai thoại và con số lịch sử trên slide không được xác nhận bởi ba tài liệu đã cung cấp.

① "Test happy path chỉ là test 1 phần"

VD: rollout sai cấu hình và trạng thái

Triển khai một endpoint mới thay thế chức năng cũ — hệ thống test đã pass "happy path" nhưng không test *ordering & state* khi gặp code mới. Một flag nhầm lẫn vẫn kích hoạt đường cũ, gây ra hàng trăm nghìn order sai. **Fix:** test phải *replay production traffic* (shadow/canary) trước khi tắt hệ cũ.

② "Đừng tin functional test cho state"

VD: request đồng thời và trùng lặp

API charge có test "đúng trong transaction", nhưng hai request *đồng thời* cùng idempotency_key khiến cả hai thành công → charge 2 lần. **Fix:** thêm concurrency test với `asyncio.gather` cho mọi endpoint nhận state mutation.

③ "Version cũ vẫn chạy với payload mới"

VD: parser và dữ liệu đầu vào thay đổi

Sau khi đẩy regex mới, parser cũ vẫn *nhận* payload và xử lý sai — không có *schema contract* nào enforce. **Fix:** validate payload bằng schema *trong code*, không chỉ ở gateway.

④ "Mock giống thật nhưng test vẫn xanh"

VD: mock không phản ánh schema thực

Test mock database trả về đúng field — production dùng schema thật có field bị xoá; trả về none và crash view. **Fix:** dùng *test container* (testcontainers-python) hoặc snapshot schema thật cho unit test của business logic.

⑤ "SLO trên dashboard ≠ SLO cho user"

VD: latency backend khác trải nghiệm người dùng

Đặt SLO `p95 < 100ms` cho search; p95 backend = 80ms nhưng search mất 2s do client render. **Fix:** SLO phải đo *user-perceived* (từ client), không chỉ service-side.

Anti-patterns tổng hợp

7 lỗi thường gặp khi viết API test

Mỗi lỗi kèm theo *triệu chứng*, *gốc rễ* và *phương pháp sửa*. Catalogue này đọc lại trước khi PR review.

① Test "happy path toàn bộ"

Gọi 1 endpoint đúng → assert 200. **Sửa:** thêm 4xx/5xx, biên, concurrent.

② Dùng lại data global

Test tạo user*id=1*, test khác đọc *id=1*. **Sửa:** mỗi test tạo data riêng qua fixture.

③ Gọi endpoint thật để "warm up"

Test gửi POST rồi mới test tiếp. **Sửa:** dùng fixture, tách dependency.

④ Test xanh vì assert

`assert response.json` không dùng method. **Sửa:** luôn `.get_json()`.

⑤ Sleep cứng thay vì chờ event

`time.sleep(2)` flaky khi CI chậm. **Sửa:** poll với timeout.

⑥ Test kết nối vào production DB

Fixture trỏ Postgres dev — xoá nhầm khó phục hồi. **Sửa:** SQLite memory / testcontainer.

⑦ Không test "unhappy Create" — happy path quen rồi

Body rỗng, body dài 1MB, body có emoji không hỗ trợ, body có thẻ script — đa số bỏ sót. **Sửa:** 10–15 payload độc hại + test từng cái.

Checklist — khi viết 1 endpoint mới

Mức ① — Functional

Mỗi endpoint phải có

- Status code cho happy path & từng error path
- Schema check trên response (đủ field, đúng type)
- Validation cho cả *thiếu field* và *field không hợp lệ*
- Test 401 (chưa auth) và 403 (không đủ quyền) riêng biệt
- Idempotency cho PUT / DELETE (gọi 2 lần)
- 404 cho resource không tồn tại

```
Số test phụ thuộc mức rủi ro và hành vi của endpoint
```

Mức ② — Contract

API thay đổi shape

- OpenAPI spec được commit cùng code
- Spectral lint pass trong CI
- Server validate request theo schema
- Mỗi thay đổi breaking có ADR / deprecation note
- Cân nhắc Pact khi có nhiều consumer cần bảo vệ kỳ vọng
- Thời gian hỗ trợ tương thích theo chính sách versioning

```
Mỗi PR thay đổi schema → re-run schema test cho tất cả consumer
```

Mức ③ — Performance

Trước khi tới production

- Load test ở mức tải dự kiến và kiểm tra SLO đã thống nhất
- Soak test đủ dài theo rủi ro rò rỉ tài nguyên
- Stress test để biết ceiling (ghi log)
- Verify zero slow query (log > 100ms vào dashboard)
- Test connection pool exhaustion
- Document decision — tại sao con số SLO chọn

```
Lưu run log + so sánh với baseline trước
```

Đọc thêm & công cụ

Tài liệu & toolchain — buổi 8

Đọc bắt buộc (đề cương)

J.J. Geewax, API Design Patterns, Manning 2021 — Chương 6: "Resource identification"

James Higginbotham, Principles of Web API Design — Chương 12: "API Testing Strategies"

Tool dùng trong buổi học

Tool	Mục đích	Cài
`pytest`	test runner cho Python, mở rộng plugin	`pip install pytest`
`pytest-flask`	fixture app, client, runner	`pip install pytest-flask`
`jsonschema`	validate JSON theo schema	`pip install jsonschema`
`@stoplight/spectral-cli`	lint OpenAPI spec	`npm i -g @stoplight/spectral-cli`
`newman`	chạy Postman collection từ CLI	`npm i -g newman`
`locust`	load test Python-native, có web UI	`pip install locust`

Test là việc của cả hệ thống — không phải của riêng QA

1. **Mỗi endpoint cần test 3 lớp**

   Protocol, Schema, Business rule. Test happy path chỉ là test 1/3.

2. **Contract + Performance mới đóng vai trò SLO gate**

   Functional pass ≠ production-ready. CI phải gồm schema diff + load test tối thiểu 1 endpoint chính.

3. **Test phải phản ánh hành vi người dùng**

   User flow có state, có auth, có random — không gọi endpoint rời rạc rồi vẽ chart đẹp.

Buổi tiếp theo, 9

API Versioning & Lifecycle

Versioning strategies, backward compatibility, deprecation. Cách làm "/v1 → /v2" mà không làm sập client.

Đọc thêm, Higginbotham Ch. 14 (Designing for Change)

Bài tập về nhà (nộp trước buổi 9)

(a) Áp dụng Postman collection cho *đồ án nhóm* của bạn; commit kèm 1 Newman script vào repo.
(b) Load test 1 endpoint chính bằng Locust; viết 1 trang *analysis note* giải thích SLO đề xuất.